

Process Scheduling Simulator

Ben Robles

Program functionality

- A simulator to test variations of work stealing algorithms for parallel processing.

Structure:

Task -> Deque -> Workers

Each worker has a deque of tasks

Workers process tasks FIFO

Processing:

Cycles -> Simulation -> Testbed

Cycles are discrete units of time.

Tasks take variable cycles to complete

Data Structures and Algorithms (1)

Each worker has a deque implemented by a doubly linked list

- Original deque was taken from the singly linked list example
- Work is stolen from the back of a victim's deque, so changing to a doubly linked list by adding a pointer to the previous node improves the speed of stealing from $O(n)$ to $O(1)$ by avoiding scanning through the deque.

```
7 typedef Task DequeElementType;
8
9 class Deque {
10 private:
11     struct Node {
12         DequeElementType data;
13         Node* next;
14         Node* prev;
```

```
// Remove an element from the rear of the deque
void Deque::pop_back() {
    if (empty()) {
        cerr << "Error: Deque is empty, cannot pop from back.\n";
        return;
    }

    if (myFront == myBack) { // If there's only one element
        delete myBack;
        myFront = myBack = nullptr;
    }
    else {
        Node* current = myFront;
        // Traverse to the second-last node
        while (current->next != myBack) {
            current = current->next;
        }
        delete myBack;
        myBack = current;
        myBack->next = nullptr;
    }
    size--;
}
```

Singly Linked List, $O(n)$

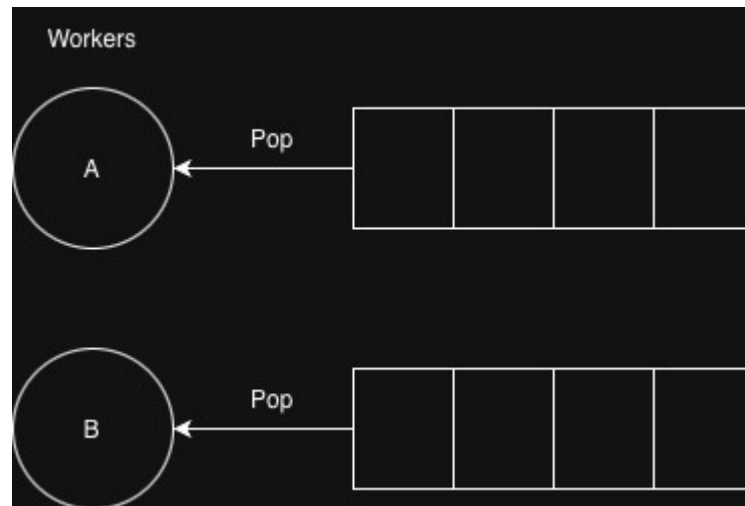
```
101 // Remove an element from the rear of the deque
102 bool Deque::steal_back(DequeElementType& value) {
103     if (empty()) {
104         return false;
105     }
106
107     value = myBack->data;
108
109     if (myFront == myBack) { // If there's only one element
110         delete myBack;
111         myFront = myBack = nullptr;
112     }
113     else {
114         Node* temp = myBack;
115         myBack = myBack->prev;
116         myBack->next = nullptr;
117         delete temp;
118     }
119     size--;
120     return true;
121 }
```

Doubly Linked List, $O(1)$

Data Structures and Algorithms (2)

No Work Stealing

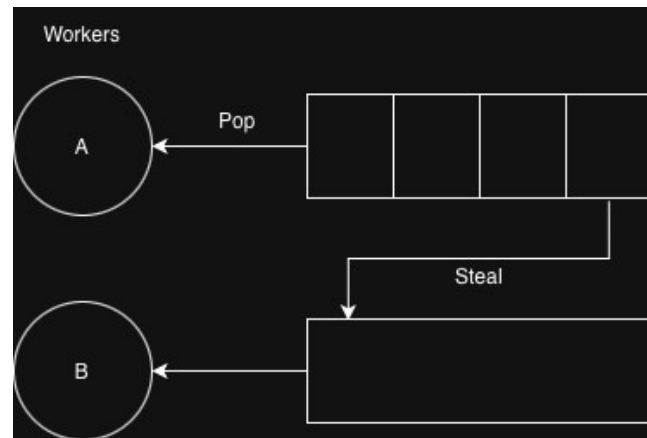
- Workers exclusively work from their local deque
- Simple baseline for comparison



Data Structures and Algorithms (3)

Idle Work Stealing

- When a worker becomes idle by having no current task and an empty deque, it attempts a steal
- Iterating through the other workers, the thief finds a worker with at least one task in its deque, steals it, and then adds it to its own deque



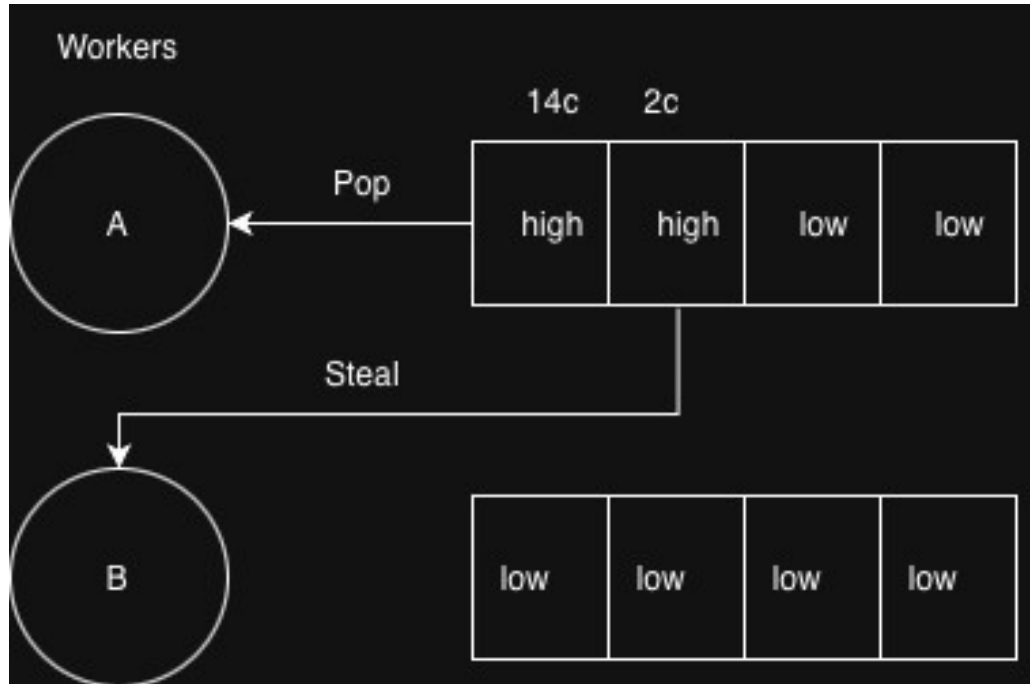
Data Structures and Algorithms (4)

Priority Work Stealing

- Each task has a priority
- If a worker doesn't have a maximum priority task it iterates through other workers deques to look for one and attempt a steal
- Each task has a type. Workers complete tasks matching their type faster (in half as many cycles)
- For each steal attempt of a maximum priority task, the worker calculates whether it's worth stealing by its type
- When no more priority tasks are left, workers default to idle stealing

Data Structures and Algorithms (4)

Priority Work Stealing



Data Structures and Algorithms (5)

- To create a dataset for an experiment, a vector of variable tasks is generated using <random>
- The vector of tasks is sorted by priority using insertion sort

```
26 → void Sort_TasksByPriority(std::vector<Task>& tasks) {  
27     for (size_t i = 1; i < tasks.size(); i++) {  
28         Task tempTask = tasks[i];  
29         size_t j = i;  
30  
31         while (j > 0 && tempTask.priority > tasks[j - 1].priority) {  
32             tasks[j] = tasks[j - 1];  
33             j--;  
34         }  
35  
36         tasks[j] = tempTask;  
37     }  
38 }
```

Potential improvements

- Implement a quicksort when the number of tasks is greater than 20
- Modify the underlying data structure to make the deque a priority deque, avoiding the need for sorting

Overview

- Each worker has a deque of tasks to process
- Processing tasks happen by cycle
- Simulation ends when all cycles are complete
- Tasks are sorted by priority
- Each task has a type and are originally assigned to the workers who will do them fastest

Testbed

- A test runs all the simulation types (No, Idle, and Priority work stealing) with the same tasks and stores its results
- Tasks are generated per test

Each test:

- 4 workers
- 50 tasks: minimum 3 cycles, maximum 40
- Maximum priority: 5 ($\approx \frac{1}{5}$ probability)

What happens after 1000 tests?

Results

```
/Users/br0bl/CLionProjects/SchedulerProject/cmake-build-debug/SchedulerProject
```

```
Running 1000 tests...
```

```
Tests where IWS decreased total cycles from NWS: 866 / 1000
```

```
Tests where PWS decreased total cycles from NWS: 813 / 1000
```

```
Tests where PWS decreased total cycles from IWS: 276 / 1000
```

```
Average cycles saved when PWS improved over IWS: 6.52174
```

```
Largest total-cycle improvement by PWS over IWS: 25
```

```
Tests where PWS took more total cycles than NWS: 166 / 1000
```

```
Tests where PWS took more total cycles than IWS: 473 / 1000
```

```
Largest total-cycle loss by PWS compared to IWS: 33
```

```
Average cycles lost when PWS was worse: 8.90909
```

```
Average cycles saved by PWS compared to IWS: -2.414
```

```
Tests where PWS decreased priority-finished cycles from baseline: 716 / 1000
```

```
Average priority cycles saved by PWS compared to baseline: 7.976
```

```
Average priority cycles saved when PWS improved priority completion: 11.1397
```

```
Largest priority-cycle improvement by PWS: 46
```

```
Average Least Cycles Possible: 134.822
```

```
Average Most Cycles Possible: 1074.98
```

```
Average NWS Total Cycles: 183.428
```

```
Average IWS Total Cycles: 162.811
```

```
Average PWS Total Cycles: 165.225
```

Key Results

- Idle work stealing finished in less cycles than no work stealing in 86.6% of tests
- Priority work stealing finished in less cycles than idle work stealing in 27.6% of tests
- Average cycles saved by priority work stealing compared to idle work stealing: -2.414

- Priority work stealing finished priority tasks in less cycles in 71.6% of tests
- Average priority cycles saved by priority work stealing: 7.976

- Average NWS Total Cycles: 183.428
- Average IWS Total Cycles: 162.811
- Average PWS Total Cycles: 165.225

Simulation Conclusion

On Average:

- Priority work stealing is 2 cycles slower than idle work stealing but finishes priority tasks 7 cycles faster
- Any form of work stealing finishes ~20 cycles faster than no work stealing
- Idle work stealing is slightly faster, but priority work stealing is more productive